AssetMon — Deployment Implementation Plan

AWS account & server setup · Database + Backend deployment · Vercel frontend connection

What we are building

The frontend (Next.js) is hosted on **Vercel**. The backend (NestJS) and the database (PostgreSQL) run together in **Docker** on a single **AWS server**. The browser loads the site from Vercel and calls the backend over HTTPS at a separate address (e.g. `api.yourco.com`). We keep PostgreSQL (no engine change).

Piece	Where it runs	Address
Frontend (website)	Vercel	app.yourco.com
Backend (API) + Database + HTTPS gateway	AWS server (Docker)	api.yourco.com

Prerequisites (have these ready)

- A domain name you control (for `app.` and `api.` subdomains).
- A credit/debit card (AWS requires one even on the free plan; a small temporary verification charge applies).
- A GitHub account with this project's code pushed to a repository (Vercel deploys from GitHub).
- The 3 strong secrets generated with `openssl rand -hex 32` (used in Part 2).

PART 1 — Create the AWS account & launch the server

1A. Create the AWS account (onboarding, step by step)

1. Open `https://aws.amazon.com` and click **Create an AWS Account**.
2. Enter your **email**, choose an **AWS account name** (e.g. "assetmon"), and verify the email with the OTP code AWS sends.
3. Set a strong **root user password**.
4. Choose account type **Personal** (or Business), then fill in your full name, phone number, country, and address.
5. Enter your **payment (card) details**. AWS may place a small temporary hold (~$1) to verify the card; it is refunded.
6. Complete **identity verification** — AWS sends an SMS or voice OTP; enter the code.
7. Choose the **Basic support plan (Free)**.
8. Account created. Click **Go to the AWS Management Console** and sign in as the root user.

■ New accounts get the **credit-based free plan** (about $100, up to $200 after a few setup activities). Confirm it under **Billing** → **Free Tier / Credits**. You can run a 4 GB server on these credits for roughly 4–6 months before paying.

1B. Secure the account & set a billing alert (do this first)

1. Enable **MFA (multi-factor authentication)** on the root user: Account menu → **Security credentials** → assign an MFA device (authenticator app).
2. Go to **Billing** → **Budgets** → **Create budget**. Create a small monthly cost budget (e.g. $1 or $5) with an **email alert** at 50% / 80% / 100%.
3. Turn on **Free Tier usage alerts** under **Billing preferences**.

✓ This guarantees you get the "credits running low — upgrade to keep running" email with plenty of lead time, so billing never surprises you.

1C. Launch the server (EC2)

1. In the console search bar, open **EC2** → **Instances** → **Launch instances**.
2. **Name:** assetmon-server.
3. **OS image (AMI):** Ubuntu Server 22.04 LTS.
4. **Instance type: t3.medium (4 GB RAM)** recommended for backend + database together. (t3.small / 2 GB is the cheaper minimum; t4g.medium is a cheaper 4 GB ARM option if you pick an ARM Ubuntu image.)
5. **Key pair:** click **Create new key pair**, download the `.pem` file, and keep it safe — it is your SSH login key.

6. **Network settings** → **firewall (security group):** allow **SSH (22)** from *My IP*, **HTTP (80)** from anywhere, and **HTTPS (443)** from anywhere.
7. **Storage:** 30 GB **gp3** SSD.
8. Click **Launch instance**. Then open the instance and note its **Public IPv4 address**.
9. (Recommended) Allocate an **Elastic IP** and associate it with the instance so the address never changes.

1D. Connect to the server (SSH)

From your computer's terminal (Windows PowerShell, macOS, or Linux):

```
# restrict the key file permissions first (macOS/Linux):
chmod 400 assetmon-key.pem

# connect (use your .pem path and the server's public IP):
ssh -i assetmon-key.pem ubuntu@<SERVER_PUBLIC_IP>
```

■ On Windows you can use the same command in PowerShell, or use PuTTY with the key converted to .ppk.

1E. Free → Paid (when credits run low)

- When the budget/credit alert email arrives, go to **Billing** and **upgrade to the paid plan** (confirm your payment method). The server keeps running with no downtime; remaining credits still apply first.
- To add more RAM later: **stop** the instance → **Actions** → **Instance settings** → **Change instance type** → pick a bigger size → **start** it again.
- If you do nothing when credits end, the credit-based plan **pauses** resources rather than charging unexpectedly — no surprise bill.

PART 2 — Deploy the Database + Backend on the server

We run PostgreSQL + the NestJS backend + a Caddy HTTPS gateway as three Docker containers on the one server.

2A. Install Docker

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
# log out and back in (or: newgrp docker) so the group applies
docker compose version   # confirm it works
```

2B. Get the project code onto the server

```
git clone <YOUR_GIT_REPO_URL> assetmon
cd assetmon
```

2C. Add the four deployment files

Create these in the project root. **apps/api/Dockerfile** (builds the backend; generates the Prisma clients and keeps the migration files):

```
FROM node:20-bookworm-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends openssl && rm -rf /var/lib/apt/lists/*
COPY package*.json ./
COPY apps/api/package.json apps/api/
COPY packages/db-catalog/package.json packages/db-catalog/
COPY packages/db-tenant/package.json packages/db-tenant/
COPY packages/shared/package.json packages/shared/
RUN npm install
COPY . .
RUN npm run db:catalog:generate && npm run db:tenant:generate
RUN npm -w @assetmon/api run build

FROM node:20-bookworm-slim AS runner
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends openssl && rm -rf /var/lib/apt/lists/*
ENV NODE_ENV=production
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json
COPY --from=builder /app/apps/api ./apps/api
COPY --from=builder /app/packages ./packages
RUN mkdir -p /app/uploads
EXPOSE 4000
CMD ["node", "apps/api/dist/main.js"]
```

docker-compose.yml (database + backend + HTTPS gateway):

```
services:
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: assetmon_catalog
    volumes: [ "pgdata:/var/lib/postgresql/data" ]
  api:
    build: { context: ., dockerfile: apps/api/Dockerfile }
    restart: unless-stopped
    env_file: .env
    depends_on: [ postgres ]
    volumes: [ "uploads:/app/uploads" ]
  caddy:
    image: caddy:2
    restart: unless-stopped
    ports: [ "80:80", "443:443" ]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: [ api ]
volumes: { pgdata: {}, uploads: {}, caddy_data: {}, caddy_config: {} }
```

Caddyfile (automatic HTTPS for the API):

```
api.yourco.com {
    reverse_proxy api:4000
}
```

.env (real secrets — never commit this file):

```
NODE_ENV=production
POSTGRES_HOST=postgres
POSTGRES_PASSWORD=<STRONG_DB_PASSWORD>
CATALOG_DATABASE_URL=postgresql://postgres:<STRONG_DB_PASSWORD>@postgres:5432/assetmon_catalog?schema=public
TENANT_DB_URL_TEMPLATE=postgresql://postgres:<STRONG_DB_PASSWORD>@postgres:5432/{db}?schema=public
JWT_ACCESS_SECRET=<openssl rand -hex 32>
JWT_REFRESH_SECRET=<openssl rand -hex 32>
ENCRYPTION_KEY=<openssl rand -hex 32>
WEB_ORIGIN=https://app.yourco.com
TRUST_PROXY_HOPS=1
UPLOAD_DIR=/app/uploads
```

■ Generate the three secrets by running `openssl rand -hex 32` three times. The backend refuses to start in production with weak/default secrets — this is intentional.

2D. Point the API domain at the server

In your domain's DNS, add an **A record**: `api.yourco.com` → your server's public IP. Caddy will then automatically obtain an HTTPS certificate on first start.

2E. Build & start everything

```
docker compose up -d --build
```

This builds the backend image and starts the database, backend, and gateway. Caddy fetches the HTTPS certificate for `api.yourco.com`.

2F. Initialise the database & create the first admin

```
# create the catalog database tables:
docker compose run --rm api npm run db:catalog:migrate

# create the first platform admin:
docker compose run --rm api npm run platform:bootstrap-admin -- --allow-prod --email you@yourco.com
```

Per-customer (tenant) databases are then created automatically the first time each workspace signs up — no manual step.

2G. (Optional) Load your existing data

If you already have data in another PostgreSQL, copy it in. Otherwise skip — the app creates fresh databases on signup.

```
# from the old DB: pg_dump each database to a .sql file, then load it in:
cat backup.sql | docker compose exec -T postgres psql -U postgres -d <database_name>
```

2H. Verify the backend is live

```
curl https://api.yourco.com/api/health
# expected: {"ok":true, ...}
```

2I. Reliability: swap file, backups, tuning

1 Add a 2 GB swap file (free safety net against memory spikes):

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

2 Schedule a **daily database backup** (a simple cron job) and copy it off the server (e.g. to S3 or download it):

```
docker compose exec -T postgres pg_dumpall -U postgres > assetmon_backup_$(date +%F).sql
```

3. Keep **one backend instance** (the app keeps state in memory). Scale by sizing the server up, not by running copies.

PART 3 — Deploy the Frontend on Vercel & connect it

3A. Push the code to GitHub

Vercel deploys from a Git repository, so make sure the project is on GitHub (the same repo can be used for the backend on the server).

3B. Create the Vercel project

1. Go to `https://vercel.com` and **Sign up with GitHub**.
2. Click **Add New…** → **Project** and **Import** your repository.
3. Set **Root Directory** to `apps/web`. Vercel auto-detects Next.js (no other build settings needed).

3C. Set the environment variables (these link it to the backend)

In the import screen (or Project → Settings → Environment Variables), add for **Production**:

Variable	Value	Why
NEXT_PUBLIC_API_URL	https://api.yourco.com	tells the website where the backend is
NEXT_PUBLIC_SITE_URL	https://app.yourco.com	the website's own public address (SEO)

■ These are read at **build time**. If you ever change the API domain, you must **redeploy** the Vercel project for it to take effect.

3D. Deploy & attach the domain

1. Click **Deploy**. Vercel builds and serves the website globally with HTTPS.
2. Go to **Project** → **Settings** → **Domains**, add `app.yourco.com`, and create the DNS record Vercel shows (usually a CNAME).

3E. Connect the two halves (the critical link)

On the **AWS server**, make sure the backend allows the Vercel website to call it. In `.env`, the `WEB_ORIGIN` must list the exact Vercel domain, then recreate the backend:

```
# .env on the server:
WEB_ORIGIN=https://app.yourco.com

# apply the change:
docker compose up -d
```

This is the CORS "guest list". Without the exact domain here, the browser blocks the website from talking to the backend.

3F. Verify end-to-end

1. Open `https://app.yourco.com` in a browser.
2. Sign up a test workspace → confirm the backend creates its database on the AWS server.
3. Log in, create an asset, upload a photo (served from `api.yourco.com`).
4. Run the project's QA scripts in `tools/` against the live API as a final check.

■ Vercel **preview** deployments get temporary URLs that are NOT on the CORS guest list, so they cannot call the production backend. Test against the real `app.yourco.com` domain.

Reference — DNS, costs & final checklist

DNS records to create

Record	Name	Points to
A	api.yourco.com	AWS server public IP
CNAME	app.yourco.com	the value Vercel gives you

Cost summary

Phase	AWS (server + DB)	Vercel (frontend)
Free period (~4-6 months on credits)	$0 (paid from credits)	$0 (Hobby plan)
After credits / production	~$24-30 / month (4 GB)	$0, or $20 / month (Pro)

Final checklist

- AWS account created, MFA on, budget + free-tier alerts set.
- EC2 server (4 GB) launched, security group opens 22/80/443, connected via SSH.
- Docker installed; project cloned; the 4 deployment files added with strong secrets.
- DNS: `api.` → server IP; `app.` → Vercel.
- `docker compose up` done; catalog migrated; first admin created; `/api/health` returns ok.
- Swap file added; daily backup scheduled.
- Vercel project deployed (root `apps/web`) with the two env vars; domain attached.
- `WEB_ORIGIN` on the server includes the Vercel domain; end-to-end signup tested.